

Cuttlefish Tier-2 A/B Evidence Report

Tier-2 Cuttlefish — REAL Android A/B update_engine OTA — Evidence Report

Revision: 1 **Last modified:** 2026-06-23T09:10:00Z **Run-id:** 20260623-cuttlefish-tier2-ab **Verdict:** PASS — REAL Android update_engine A/B + Virtual-A/B + auto-rollback, verified on a live Cuttlefish cvd with captured evidence (§11.4.5 / §11.4.69 / §11.4.107 / §11.4.108). **Authority:** §11.4.83 (per-feature e2e evidence), §11.4.3 / §11.4.112 (Cuttlefish is the hardware-free A/B proxy), §11.4.133 (bounded, inactive-slot-only destructive write), §11.4.6 (every claim is a captured FACT, no guessing), §11.4.158 (read-the-screen content verification).

1. What was proven

A REAL Android A/B over-the-air update was applied end-to-end on a live Cuttlefish virtual device (build **15660610**, aosp_cf_x86_64_only_phone, Android 17 / API 37) running on the operator's nezha Linux+KVM host, driven autonomously from that host as user milosvasic over adb -s 127.0.0.1:6520 — **no sudo for the A/B flow itself**. The three headline proofs of native A/B fidelity — real payload apply, slot flip, and auto-rollback — all reproduced with captured evidence. This is the deepest tier the Helix OTA emulation ladder reaches without physical silicon.

A/B capability of the cvd (captured, ab_facts.txt):

- ab_update=true, ro.virtual_ab.enabled=true (Virtual A/B + compression + userspace snapshots)
- veritymode=enforcing (AVB/dm-verity active)
- **15 A/B partitions:**
boot, init_boot, odm, odm_dlk, product, system, system_dlk, system_ext, vbmeta, vbmeta_system, vbmeta_system_dlk, vbmeta_vendor_dlk, vendor, vendor_boot, vendor_dlk
- bootctl HAL android.hardware.boot@aidl::IBootControl, number-slots=2
- update_engine daemon **running + responsive**
(init.svc.update_engine=running,
onStatusUpdate(UPDATE_STATUS_IDLE (0), 0))

FACT (§11.4.6): `bootctl` and `update_engine_client` work **only as root** on the `cvd` (`su / selinux u:r:su:s0`), not from a plain shell. The A/B *apply* itself is driven from the host via the AOSP `update_device.py` over `adb` and does **not** require host `sudo`.

2. Headline proof #1 — REAL OTA payload apply (`apply_full.log`, `apply.log`)

The OTA payload was obtained **autonomously, with no credentials**: the public `androidbuildinternal.googleapis.com` build API serves a **pre-signed GCS URL** (`host.storage.googleapis.com`) for `aosp_cf_x86_64_only_phone-ota-15660610.zip` — **1003473429 bytes, md5 `d90870a9a6eece3868520d7fd3f098c`** (size + md5 verified; `ota_artifact_availability.txt`, `ota_verified.txt`). The zip contains the expected `payload.bin` (1003462967 B) + `payload_properties.txt` + `care_map.pb` + metadata (`ota-type=AB`, `post-build ... /15660610`, `ota_metadata.txt`).

Applied via the AOSP `update_device.py` over `adb` (`apply.log`, `verify_only.log`):

- **Applicability check first:** `update_engine_client --verify ->`
[INFO:update_engine_client_android.cc(269)] Payload is applicable.
- **Full apply** progressed through the genuine `update_engine` state machine (`apply_full.log`): `UPDATE_STATUS_UPDATE_AVAILABLE ->`
`CLEANUP_PREVIOUS_UPDATE -> DOWNLOADING (0 -> 1.0) -> VERIFYING (0 -> 1.0) -> FINALIZING (0 -> 1.0) ->`
- **Completion (verbatim):**
[INFO:update_engine_client_android.cc(102)]
`onPayloadApplicationComplete(ErrorCode::kSuccess (0))`
[INFO:update_engine_client_android.cc(94)]
`onStatusUpdate(UPDATE_STATUS_UPDATED_NEED_REBOOT (6), 0)`
`INFO:root:Update took 115.394 seconds`

`kSuccess + UPDATED_NEED_REBOOT` = the payload was written to the inactive slot and the slot was armed. Not a metadata check, not a dry run — a real ~1 GB payload through the real daemon to a real inactive slot.

3. Headline proof #2 — slot flip `_a -> _b` (`slot_flip.log`, `slot_before.txt`, `slot_after.txt`)

After the apply, a reboot was performed and the **running active slot flipped from `_a` to `_b`**:

```
== SLOT-FLIP TEST 2026-06-23T08:53:41Z ==  
SLOT_BEFORE(running)=_a active-next=1
```

```
rebooting...
SLOT_AFTER(running)=_b
bootctl-current-after=1
snapshot-merge-status-after=merging
RESULT: SLOT FLIPPED _a -> _b (REAL A/B SLOT SWITCH CONFIRMED)
```

- slot_before.txt = _a, slot_after.txt = _b (runtime ro.boot.slot_suffix).
- bootctl current-slot = 1 (= _b) post-boot.
- Virtual-A/B merge transitioned merging -> none and slot _b was marked **successful** (the userspace snapshot merge completing on the new slot — the genuine VAB path, not legacy A/B).

This is a REAL bootloader-level A/B slot switch onto the freshly-applied slot.

4. Headline proof #3 — auto-rollback (rollback.log, rollback_trace.txt, corrupt_dd.txt)

With _b known-good and active, slot _a was deliberately made un-bootable and the device was forced to try it — and **the device rejected bad _a and booted known-good _b**:

```
== AUTO-ROLLBACK TEST 2026-06-23T08:55:16Z ==
GOOD_SLOT(running,successful)=_b
-- corrupting inactive slot _a (bounded write to boot_a, inactive-
only) --
corrupt rc=0 (see corrupt_dd.txt)
-- forcing next boot to corrupted slot _a (bootctl set-active-
boot-slot 0) --
set-active rc=0 -> active-next=0
-- reboot, expecting AUTO-ROLLBACK to known-good _b --
BOOTED=1  SLOT_AFTER_ROLLBACK=_b
RESULT: AUTO-ROLLBACK CONFIRMED -- forced-bad slot _a rejected,
device booted known-good _b
```

- **Corruption was bounded + inactive-slot-only (§11.4.133):** a **256 KB** write to boot_a (the *inactive* slot — never the active/good slot), corrupt_dd.txt: 262144 bytes (256 K) copied. The active known-good _b was never touched.
- _a was marked unbootable AND set active-next (set-active rc=0 -> active-next=0) to force the bad-boot path.
- After reboot, slot_after_rollback.txt = _b, bootctl current-slot=1 — the device **auto-rolled-back** to the known-good slot. rollback_trace.txt captures the boot trace.

This is the headline Tier-2 proof: a corrupted slot is rejected and the device recovers to the last-known-good slot, exactly as a fielded A/B device must.

5. OTA-payload resolution (no-credentials, autonomous)

The single hardest part of an autonomous A/B test — getting a real signed OTA payload without operator credentials — was solved (ota_artifact_availability.txt):

- androidbuildinternal.googleapis.com lists the build's artifacts **with no auth**, including aosp_cf_x86_64_only_phone-ota-15660610.zip.
- Requesting its download yields a **pre-signed GCS URL** (host=storage.googleapis.com, query-params present) — **no credentials required**.
- The download was integrity-checked (size 1003473429 + md5 d90870a9a6eece3868520d7fd3f098c) before apply.

The full journey to this point (provenance): curl download failed -> resumable wget -c recovery -> 27.6 GB single-stage image attempt -> slimmed to a 1.11 GB prebuilt-deb path (containers submodule 54aa9b2) -> operator privileged launch (cf-launch.sh) -> cvd booted -> this A/B PASS. The cvd was left running on nezha.

6. Honest boundary (§11.4.3 / §11.4.112 / §11.4.133)

- **Cuttlefish is the hardware-free A/B proxy.** This PASS proves native Android update_engine A/B + Virtual-A/B + AVB/dm-verity + auto-rollback **on a virtual cvd**, the deepest fidelity reachable without silicon. It is NOT a claim about any specific physical board's bootloader.
 - **The RK3588 / Orange Pi 5 Max boards stay control-plane-only by operator decision (§11.4.133).** Both fielded boards are NON-A/B (single-slot, no update_engine); their validated surface is the device-originated control-plane round-trip (F113, docs/qa/20260622-rk3588-controlplane/). Native A/B apply is **Cuttlefish-only by operator decision** — not run on the boards.
 - **bootctl/update_engine_client are root-only on the cvd** (FACT, §1) — the A/B flow is driven from the host via update_device.py over adb; only the in-guest control commands need su.
 - **Determinism:** this is a single captured full-journey run (apply -> flip -> rollback), each step with its own captured artefact. The standing §11.4.135 regression guard (tests/regression/guard_cuttlefish_ab_proven.sh) asserts these evidence anchors on every sweep.
-

7. Read-the-screen content verification (§11.4.158)

Each headline claim was machine-read against its captured artefact (not “a log exists” — the exact verbatim line was confirmed present):

#	Claim	Evidence file	Verbatim line read + confirmed
1	A/B + VAB +	ab_facts.txt	

#	Claim	Evidence file	Verbatim line read + confirmed
	verity capability		ab_update=true / virtual_ab=true / veritymode=enforcing / 15 ab_ota_partition IBootControl
2	Payload applicable	verify_only.log	Payload is applicable.
3	Apply success	apply_full.log	onPayloadApplicationComplete(ErrorCo (0)) -> UPDATE_STATUS_UPDATED_NEED_RE Update took 115.394 seconds
4	Slot flip _a -> _b	slot_flip.log + slot_before.txt/slot_after.txt	RESULT: SLOT FLIPPED _a -> _b (REAL SWITCH CONFIRMED) / _a -> _b
5	Bounded inactive-only corruption	corrupt_dd.txt	262144 bytes (256 K) copied (boot_a, in
6	Auto-rollback	rollback.log + slot_after_rollback.txt	RESULT: AUTO-ROLLBACK CONFIRMED -- f slot _a rejected, device booted know
7	OTA payload no-creds + integrity	ota_artifact_availability.txt / ota_verified.txt	host=storage.googleapis.com pre-signed 1003473429 / md5: d90870a9a6eece3868

Overall verdict: PASS — REAL Android A/B OTA, slot flip, and auto-rollback proven on a live Cuttlefish cvd with captured, read-the-screen-verified evidence.

8. Evidence manifest

File	Content
ab_facts.txt	cvd A/B/VAB/verity capability snapshot (root bootctl + update_engine)
ota_artifact_availability.txt / ota_availability.txt	no-creds androidbuildinternal API artifact list + pre-signed GCS URL + size/md5
ota_verified.txt / ota_metadata.txt	downloaded OTA zip integrity + payload contents + AB metadata
verify_only.log	Payload is applicable. applicability proof
apply.log / apply_full.log	full update_engine apply state machine -> kSuccess -> UPDATED_NEED_REBOOT
slot_before.txt / slot_after.txt / slot_flip.log	slot flip _a -> _b runtime + bootctl + VAB merge
corrupt_dd.txt	bounded 256 KB inactive-slot boot_a corruption (§11.4.133)

File	Content
rollback.log / slot_after_rollback.txt / rollback_trace.txt	forced-bad-slot reboot -> auto-rollback to known-good _b
dm_slot_layout.txt / verity_after.txt	partition/dm/verity layout
update_engine_present.txt / update_engine_follow.txt	daemon presence + follow trace

Driven from nezha as milosvasic, adb -s 127.0.0.1:6520. cvd left running.